Project Delivery Methodology (PDM)

System Requirements

[Graduation Project 95]

[SmartClinic System “Multi-Tenant”]

VERSION: [2.0] REVISION DATE: [15-3-2026]

Approval of the System Requirements indicates an understanding of the purpose and content described in this deliverable. By signing this deliverable, each individual agrees with the content contained in this deliverable.

Approver Name	Title	Signature	Date

Revision History

Version	Date	Name	Description
1.0	12/3/2026	Abdelrahman Khalawy	Initial Creation
1.1	13/3/2026	Abdelrahman Khalawy	Add Some Sections
2.0	15/3/2026	Abdelrahman Khalawy	Multi-Tenant

Contents

Section 1 Purpose 3
Section 2 General System Requirements 3
2.1 Major System Capabilities 3
2.2 Major System Conditions 3
2.3 System Interfaces 3
2.4 System User Characteristics 4
Section 3 Security Requirements 4
Section 4 Initial Capacity Requirements 4
Section 5 Initial System Architecture 5
Section 6 Use Case Diagram 5
Section 7 Database Design (ERD) 5
Section 8 System Modules / APIs 6
Section 9 Project Structure 6
Section 10 Deployment 6
Section 11 System Acceptance Criteria 7
Section 12 Current System Analysis 7
Section 13 References 7

Section 1 Purpose

The purpose of this document is to define the system requirements for the **SmartClinic Multi-Clinic System**, a web-based application supporting multiple clinics (tenants). Each clinic has its own doctors, receptionists, patients, and appointments, while using a centralized platform. The document describes functional and non-functional requirements, system architecture, security, and acceptance criteria.

Section 2 General System Requirements

2.1 Major System Capabilities

- Support multiple clinics (tenants) with data isolation per clinic.
- Secure login and registration for all users.
- Role-based access: Super Admin, Clinic Admin, Receptionist, Doctor, Patient.
- Receptionists manage patient records and appointments.
- Doctors view schedules, patient records, add diagnoses/prescriptions.
- Patients register, view doctors, and book/cancel appointments.
- Dashboard with clinic-specific statistics.
- Web-based interface accessible via desktop or mobile.
- Centralized database with TenantID-based data separation.

2.2 Major System Conditions

- Developed using **ASP.NET Core** and **Entity Framework Core**.
- Database: **Microsoft SQL Server**.
- Web-based accessible via modern browsers (Chrome, Edge, Firefox).
- Only authorized users access clinic data.
- Internet access optional for internal clinic deployment.

⇒

2.3 System Interfaces

- **Web Browser Interface:** Chrome, Edge, Firefox.
- **Database Interface:** SQL Server.
- **Server Interface:** IIS or cloud hosting.
- **Application Interface:** HTTP requests between frontend and backend.

⇒

2.4 System User Characteristics

Roles & Responsibilities:

Role	Responsibilities	Device
Super Admin	**Manage all clinics, users, and reports**	**Desktop**
Clinic Admin	**Manage doctors, receptionists, and clinic data**	**Desktop**
Receptionist	**Add/edit patients, schedule/cancel appointments**	**Desktop**
Doctor	**View schedule, patient info, add diagnoses/prescriptions**	**Desktop**
Patient	**Register, view doctors, book/cancel appointments**	**Mobile / PC**

⇒

Section 3 Security Requirements

- Authentication with username/password.
- Role-Based Access Control (RBAC) per role.
- Password encryption and secure storage.
- Tenant-based data access to ensure users access only their clinic data.
- Session expiration after inactivity.
- Protection against unauthorized access or manipulation.

⇒

Section 4 Initial Capacity Requirements

- 10–50 staff users per clinic (Admins, Receptionists, Doctors).
- 1000–5000 patients per clinic initially.
- 50–200 appointments per day.
- Response time <2 seconds for standard operations.

⇒

Section 5 Initial System Architecture

- **Architecture Layers:**

  1. **Presentation Layer:** HTML, CSS, Bootstrap
  2. **Business Logic Layer:** ASP.NET Core C#
  3. **Data Access Layer:** Entity Framework Core, SQL Server

- **Multi-Tenant Support:**

  1. All tables include **ClinicId** for tenant separation.

- **Hardware/Software Requirements:**

  1. Standard PC: i5, 8GB RAM, 256GB SSD
  2. Visual Studio, SSMS, IIS / Cloud hosting
  3. Windows OS

  ⇒

Section 6 Use Case Diagram

Actors: Super Admin, Clinic Admin, Receptionist, Doctor, Patient
Main Use Cases:

- Login / Logout
- Manage Clinics (Super Admin)
- Manage Users (Doctors, Receptionists)
- Manage Patients (Receptionists)
- Schedule / Edit / Cancel Appointments
- View Dashboard (clinic statistics)

Usecase diagram design

⇒

Section 7 Database Design (ERD)

Table	Key Fields	Notes
Clinics	ClinicId, Name, Address, Phone, Email	Tenant identifier
Users	UserId, Name, Email, Password, Role, ClinicId	Staff or Patient
Roles	RoleId, Name	SuperAdmin, ClinicAdmin, Receptionist, Doctor, Patient
Doctors	DoctorId, Name, Specialization, ClinicId	FK to Clinic
Receptionists	UserId, ClinicId	FK to Clinic
Patients	PatientId, Name, Contact, ClinicId	FK to Clinic
Appointments	AppointmentId, DoctorId, PatientId, ClinicId, Date, Time	FK to Doctor/Patient/Clinic

Notes:

- ClinicId enforces tenant-based data isolation.
- Receptionist is a user role with ClinicId for access to only their clinic’s patients and appointments.

ERD Diagram

⇒

Section 8 System Modules / APIs

1. **Authentication Module** – Login, Register, Role management.
2. **Clinic Management Module** – Add/Edit/Delete Clinics (Super Admin).
3. **User Management Module** – Add/Edit/Delete Doctors, Receptionists (Clinic Admin).
4. **Patient Management Module** – Add/Edit/Delete/View Patients (Receptionist).
5. **Appointment Module** – Schedule/Edit/Cancel Appointments.
6. **Dashboard Module** – Clinic-specific statistics, daily overview.

⇒

Section 9 Project Structure

The SmartClinic project follows a layered architecture.

```
SmartClinic
│
├── Controllers
│  ├── AuthController
│  ├── ClinicController
│  ├── PatientController
│  ├── DoctorController
│  └── AppointmentController
│
├── Models
│  ├── User
│  ├── Patient
│  ├── Doctor
│  ├── Clinic
│  └── Appointment
│
├── Data
│  └── ApplicationDbContext
│
├── Services
│  └── TenantService
│
└── Views
```

⇒

Section 10 Deployment

- Publish ASP.NET Core app → Configure IIS → Setup SQL Server → Apply EF Core Migrations → Launch via browser

- Optional: Cloud deployment (Azure)

⇒

Section 11 System Acceptance Criteria

- Users can login/register successfully.
- Role-based access enforced.
- Tenant-based data isolation (clinic-level) works.
- Authorized users can CRUD patients, doctors, appointments.
- Dashboard shows clinic-specific stats.
- System responsive (<2s) and error-free.

⇒

Section 12 Current System Analysis

Currently, there is no automated system used for managing clinic operations. Most small clinics rely on manual methods such as paper records or basic spreadsheet files to manage patient information, doctor schedules, and appointments.

These manual processes may lead to several challenges such as data loss, scheduling conflicts, and inefficient record management.

The SmartClinic system aims to replace these manual processes by providing a centralized digital platform for managing patient records, doctor information, and appointment scheduling efficiently.

No existing system integration or data conversion is required for the initial implementation of the system.

⇒

Section 13 References

Provide a list of all documents and other sources of information referenced in this document and utilized in its development. Include for each the document number, title, date, and responsible office/author.

Document No.	Document Title	Date	Author